

Informe Técnico y Análisis de Código: Motor de Juego Sweet Crush en C++ mediante Manipulación de Bits y Memoria Dinámica

Juan Camilo Restrepo Cepeda
Universidad de Antioquia

21 de septiembre de 2026

1. Introducción y Requerimientos de Bajo Nivel

El objetivo de este proyecto es construir el motor lógico del juego Sweet Crush bajo restricciones severas de programación orientada a sistemas. El diseño se distancia por completo de la abstracción orientada a objetos convencional, prohibiendo el uso de estructuras (`struct`), clases (`class`), la librería estándar de plantillas (STL) y objetos de texto (`std::string`).

Toda la representación del tablero de juego reside en un único puntero a memoria dinámica de tipo `unsigned char*`. Puesto que cada casilla ocupa exactamente 3 bits, el sistema requiere una capa de direccionamiento bit a bit para interpretar las coordenadas (f, c) de la cuadrícula sobre un vector unidimensional contiguo de bytes.

2. Análisis de la Estructura de Datos y Distribución de Bits

2.1. Codificación del Tablero

Cada posición del tablero se representa con una combinación binaria de 3 bits, permitiendo almacenar hasta 8 estados diferentes:

- 000 a 101 (0 a 5): Fichas jugables (`FICHA_A` a `FICHA_F`).
- 110 (6): Estado `VACIO`, casillas pendientes de recibir gravedad.
- 111 (7): Estado `FUERA_TABLERO`, para marcar zonas no jugables.

2.2. Matemática de Posicionamiento

Para un tablero con F filas y C columnas, una celda en la fila f y columna c tiene un índice lineal $I = f \times C + c$. Su posición en la memoria física se calcula mediante:

1. **Bit global de inicio:** $bit_inicio = I \times 3$

2. **Byte físico inicial:** $byte_inicio = \lfloor bit_inicio / 8 \rfloor$
3. **Desplazamiento interno (Offset):** $offset = bit_inicio$

3. Análisis Detallado del Código Fuente

En esta sección se examinan las funciones más críticas de la implementación, analizando cómo resuelven la manipulación de memoria y la lógica del juego.

3.1. Acceso a Bajo Nivel: Lectura y Escritura de Bits

La función `leerCelda` extrae los 3 bits correspondientes a una posición del tablero, resolviendo el problema de las celdas que se encuentran divididas entre dos bytes consecutivos.

```
unsigned char leerCelda(const unsigned char* tablero,
                        int fila, int col, int numCols) {
    int indice = fila * numCols + col;
    int bitInicio = indice * 3;
    int byteInicio = bitInicio / 8;
    int offset = bitInicio % 8;

    // Ventana auxiliar de 16 bits para unificar bytes adyacentes
    unsigned short ventana = tablero[byteInicio];
    if (offset + 3 > 8) {
        ventana |= (tablero[byteInicio + 1] << 8);
    }

    // Aislamiento de los 3 bits deseados
    return (ventana >> offset) & 0x07;
}
```

Explicación del código:

- **Cálculo de índice y offset:** Convierte la coordenada 2D a la posición absoluta dentro de la secuencia de bits.
- **Uso de ventana de 16 bits:** Carga el byte donde inicia la ficha en una variable de tipo `unsigned short`. Si la ficha sobrepasa el límite del byte ($offset + 3 > 8$), desplaza el byte siguiente 8 bits a la izquierda y lo une a la ventana con un operador OR (`|`).
- **Desplazamiento y Máscara:** Mueve los bits hacia la derecha según el `offset` para alinearlos al bit 0, y aplica la máscara `0x07` (binario `00000111`) con la operación AND (`&`) para ignorar los bits restantes.

Por su parte, la función `escribirCelda` permite modificar los 3 bits de una celda sin alterar la información de las fichas vecinas:

```

void escribirCelda(unsigned char* tablero, int fila, int col,
                  int numCols, unsigned char valor) {
    int indice = fila * numCols + col;
    int bitInicio = indice * 3;
    int byteInicio = bitInicio / 8;
    int offset = bitInicio % 8;

    // Máscara de limpieza para 3 bits
    unsigned short mascaraLimpia = ~(0x07 << offset);
    unsigned short nuevoValor = (valor & 0x07) << offset;

    if (offset + 3 <= 8) {
        tablero[byteInicio] &= mascaraLimpia;
        tablero[byteInicio] |= nuevoValor;
    } else {
        unsigned short ventana = tablero[byteInicio] |
                                (tablero[byteInicio + 1] << 8);
        ventana &= mascaraLimpia;
        ventana |= nuevoValor;
        tablero[byteInicio] = ventana & 0xFF;
        tablero[byteInicio + 1] = (ventana >> 8) & 0xFF;
    }
}

```

3.2. Modificación Estructural: Desplazamiento de Celdas

Cuando se inserta una fila, es necesario desplazar todas las celdas posteriores a la derecha. Para evitar la pérdida de datos por solapamiento, el recorrido debe realizarse en orden inverso:

```

void desplazarCeldasDerecha(unsigned char* tablero, int totalCeldas,
                           int inicioIndice, int cantidad) {
    // Recorrido inverso para no sobrescribir celdas no leídas
    for (int i = totalCeldas - 1; i >= inicioIndice; i--) {
        int fOrigen = i / numCols;
        int cOrigen = i % numCols;
        unsigned char valor = leerCelda(tablero, fOrigen, cOrigen, numCols);

        int iDestino = i + cantidad;
        int fDestino = iDestino / numCols;
        int cDestino = iDestino % numCols;
        escribirCelda(tablero, fDestino, cDestino, numCols, valor);
    }
}

```

Explicación del código: El bucle inicia desde la última celda del tablero (`totalCeldas - 1`) y retrocede hasta la posición de inserción (`inicioIndice`). En cada iteración, extrae

el valor original con `leerCelda` y lo escribe `cantidad` posiciones más adelante. Este enfoque garantiza que la casilla destino ya haya sido procesada y leída previamente.

3.3. Lógica de Juego: Gravedad y Cascada

El mantenimiento de la dinámica de juego se basa en hacer descender las fichas suspendidas sobre casillas vacías:

```
void aplicarGravedad(unsigned char* tablero, int filas, int cols) {
    for (int c = 0; c < cols; c++) {
        int escritura = filas - 1;

        // Recorrido de abajo hacia arriba en la columna
        for (int f = filas - 1; f >= 0; f--) {
            unsigned char v = leerCelda(tablero, f, c, cols);
            if (v != VACIO && v != FUERA_TABLERO) {
                if (f != escritura) {
                    escribirCelda(tablero, escritura, c, cols, v);
                    escribirCelda(tablero, f, c, cols, VACIO);
                }
                escritura--;
            }
        }
    }
}
```

Explicación del código: Se utiliza una técnica de dos punteros lógicos por cada columna: el índice `f` explora las celdas de abajo hacia arriba, mientras que `escritura` apunta a la posición más baja disponible para ubicar una ficha válida. Si se detecta una ficha jugable en la posición `f`, se mueve a la posición `escritura` y la casilla superior original se marca como `VACIO`.

4. Gestión Dinámica de Memoria y Umbral del 65 %

Para optimizar el uso de recursos, el tablero administra dinámicamente el tamaño de su reserva en memoria. Cuando se ejecutan eliminaciones de filas o columnas, la memoria física reservada no se comprime inmediatamente si la cantidad de bytes requeridos sigue siendo cercana a la capacidad actual.

La reducción de memoria física únicamente se ejecuta cuando el uso real cae por debajo del 65 % respecto a la capacidad reservada:

$$Porcentaje\ de\ uso = \left(\frac{Bytes\ necesarios}{Bytes\ reservados} \right) \times 100$$

Si la condición se cumple, se reasigna el buffer de memoria utilizando un puntero temporal, se copian los datos activos compactados y se libera la memoria previa con `delete[]`.

5. Estructura Modular del Sistema

El código se distribuye en módulos con responsabilidades aisladas:

- `bits.cpp`: Abstracción de bajo nivel (máscaras, desplazamientos).
- `tablero.cpp`: Asignación, liberación y reconstrucción de buffers de memoria.
- `filas_columnas.cpp`: Operaciones topológicas sobre el arreglo continuo.
- `juego.cpp`: Motor de reglas (detección de trios, puntuación, cascadas).
- `vista.cpp`: Salida en formato consola e inspección binaria.

6. Conclusiones

La implementación de Sweet Crush demuestra que el uso estricto de punteros dinámicos y la manipulación de datos a nivel de bits permiten construir sistemas complejos manteniendo un consumo mínimo de memoria. El aislamiento de la lógica de acceso en funciones como `leerCelda` y `escribirCelda` resolvió el desafío de manipular elementos no alineados a los límites estándar del byte, garantizando modularidad y estabilidad durante la ejecución.